BÁO CÁO THỰC HÀNH LAB 14

NETWORK TROUBLESHOOTING

Người báo cáo: Liamz07

1. Mục tiêu bài thực hành:

- **Làm chủ quy trình chẩn đoán sự cố mạng (Troubleshooting Methodology):** Áp dụng tư duy logic để khoanh vùng và tìm ra nguyên nhân gốc rễ của lỗi trong mô hình kiến trúc phân tầng.
- **Thành thạo mô hình Reverse Proxy:** Quản lý cơ chế tương tác giữa Client → Nginx Reverse Proxy (Cổng :80) → Node.js Backend (Cổng :3000).
- **Sử dụng bộ công cụ quản trị Linux & PM2:** Khai thác hiệu quả các câu lệnh curl, ss, pm2, và nhật ký hệ thống (logs) để kiểm tra từ tầng ứng dụng đến tiến trình dịch vụ.

2. Thông tin môi trường và kịch bản thực hành:

- **Mô hình kiến trúc triển khai:** Client → Nginx (:80) → Node.js Backend (:3000)
- **Kịch bản sự cố cố tình (Intentional Fault):** Dịch vụ Node.js (Backend) không chạy dẫn đến việc Nginx không thể chuyển tiếp yêu cầu, gây ra lỗi 502 Bad Gateway cho Client.

3. Quy trình từng bước chẩn đoán và khắc phục sự cố (Step-by-Step Troubleshooting):

BƯỚC 1: KIỂM TRA PHẢN HỒI TỪ TẦNG ỨNG DỤNG BẰNG CÔNG CỤ CURL

- **Mục đích:** Kiểm tra xem dịch vụ Web chính (Nginx ở cổng 80) có phản hồi yêu cầu từ client hay không và xác định mã lỗi HTTP trả về.
- **Lệnh thực thi:** curl -I http://<IP_Server>
- **Kết quả quan sát & Phân tích:**
  - *Phản hồi trả về mã lỗi HTTP/1.1 502 Bad Gateway.*
  - *Giải thích:* Nginx hoạt động tốt và nhận request từ client trên cổng 80, nhưng do ứng dụng phía sau (Node.js ở cổng 3000) đang tắt, Nginx không thể kết nối tới upstream và trả về lỗi 502.

BƯỚC 2: KIỂM TRA TRẠNG THÁI CỔNG MẠNG VÀ SOCKET BẰNG CÔNG CỤ SS

- **Mục đích:** Xác định xem các cổng dịch vụ :80 và :3000 có đang ở trạng thái lắng nghe (LISTEN) hay không.
- **Lệnh thực thi:** sudo ss -tulpn | grep -E ':80|:3000'
- **Kết quả quan sát & Phân tích:**
  - *Tại thời điểm xảy ra sự cố, hệ thống chỉ hiển thị tiến trình lắng nghe trên cổng 80 (của Nginx), hoàn toàn không có cổng 3000.*
  - *Kết luận:* Tiến trình Backend chưa hoạt động, socket chờ kết nối chưa được mở.

BƯỚC 3: KIỂM TRA TRẠNG THÁI TIẾN TRÌNH ỨNG DỤNG BẰNG CÔNG CỤ QUẢN LÝ PM2

- **Mục đích:** Rà soát danh sách tiến trình Node.js được quản lý bởi công cụ PM2 trên hệ thống.
- **Lệnh thực thi:** pm2 status
- **Kết quả quan sát & Phân tích:**
  - *Kiểm tra thư mục chứa mã nguồn (ls -la thấy tệp app.js) và trạng thái PM2 cho thấy tiến trình web-app đang ở trạng thái stopped hoặc chưa được khởi chạy.*
  - *Kết luận:* Xác thực nguyên nhân gốc rễ là do tiến trình Node.js quản lý qua PM2 chưa khởi động.

BƯỚC 4: PHÂN TÍCH NHẬT KÝ HỆ THỐNG VÀ LỖI ỨNG DỤNG BẰNG PM2 LOGS

- **Mục đích:** Đọc nhật ký lỗi (error logs) từ tiến trình và máy chủ web để ghi nhận vết lỗi kết nối.
- **Lệnh thực thi:** pm2 logs web-app --lines 50
- **Kết quả quan sát & Phân tích:** Nhật ký ghi nhận lịch sử dừng tiến trình và kiểm tra log lỗi Nginx (/var/log/nginx/error.log) hiển thị thông báo: connect() failed (111: Connection refused) while connecting to upstream.

BƯỚC 5: KHẮC PHỤC SỰ CỐ (REMEDIATION)

- **Phương án xử lý:** Tiến hành khởi động lại ứng dụng Node.js (app.js) thông qua trình quản lý tiến trình PM2 (pm2 start app.js).
- **Kết quả ghi nhận:**
  - *Lệnh pm2 status trả về trạng thái online (màu xanh lá), PID hoạt động và tiêu thụ tài nguyên ổn định.*
  - *Lệnh kiểm tra log (pm2 logs web-app) hiển thị thông báo thành công: Server running at http://0.0.0.0:3000/.*

BƯỚC 6: KIỂM TRA XÁC THỰC LẠI HỆ THỐNG SAU KHI KHẮC PHỤC

- **Kiểm tra cổng mạng bằng ss:** sudo ss -tulpn | grep -E ':80|:3000'
  - *Kết quả:* Hiển thị đồng thời cổng 80 (nginx) và cổng 3000 (node) đều ở trạng thái LISTEN.
- **Kiểm tra HTTP bằng curl:** curl -I http://localhost
  - *Kết quả:* Phản hồi trả về mã HTTP/1.1 200 OK, xác nhận toàn bộ mô hình Client → Nginx (:80) → Node.js (:3000) đã hoạt động thông suốt trở lại.

4. Kết quả và bài học rút ra:

- **Hoàn thành xuất sắc bài Lab 14:** Nắm vững quy trình chẩn đoán lỗi từ tầng ứng dụng, quét cổng mạng đến việc kiểm tra trình quản lý tiến trình thực tế (PM2).

- **Nâng cao kỹ năng quản trị Linux:** Sử dụng linh hoạt bộ công cụ chẩn đoán chuyên nghiệp (curl, ss, pm2 logs) để xử lý các sự cố hạ tầng web thực tế.

HẾT!